Kopilkin

Microservices, Data, Routing, Infrastructure and Recommender System Report

---

Student Name

Mubarakov Islam

Group

11-314a

Assignment

3rd task series (2nd semester)

GitHub Repository

https://github.com/iz-mv/Kopilkin

Task Block 3 - Microservices, Data, Routing, Infrastructure and Recommender System Report

Kopilkin Personal Finance Application

Mubarakov Islam

1. Project overview

Kopilkin is a mobile-first personal finance web application. Users can register, log in, add income and expense transactions, create savings goals, ask a multi-agent AI assistant for spending analysis, and receive smart financial recommendations. In Task Block 3, the project was extended from a basic MVP into a microservice-oriented system with Kafka events, PostgreSQL databases, Redis caching, an API Gateway, Nginx load balancing, and a recommender system.

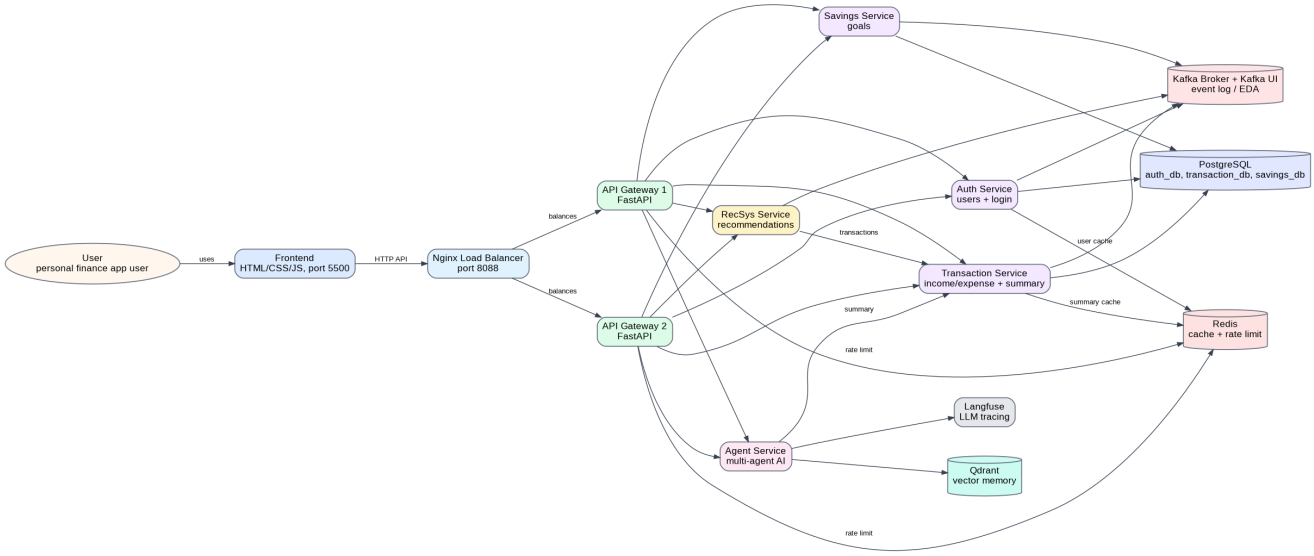

Figure 1. Updated Block 3 architecture overview.

2. Scope of Task Block 3

This report covers the whole third task series, not only C4 diagrams. The project implementation and documentation are mapped to requirements 5-11 below.

Requirement	Implemented result	Main evidence in the project
5. Communication via Kafka / EDA	Kafka broker, Kafka UI and event producers were added. Services publish domain events after business operations.	docker-compose.yml; app/events.py in auth, transaction, savings and recsys services.
6. Data: RDBMS + NoSQL / cold storage	Business data moved to PostgreSQL databases. Qdrant stores vector memory. Langfuse uses its own PostgreSQL database. Kafka works as event log; Redis stores short-lived data.	infra/postgres/init.sql; service database.py and models.py files; qdrant/langfuse containers.
7. Redis-like caching	Redis is used for user profile cache, transaction summary cache and rate limiting counters.	auth-service/app/cache.py; transaction-service/app/cache.py; api-gateway/app/main.py.
8. Routing layer	Nginx load balancer routes traffic to two API Gateway instances. API Gateway routes requests to internal services and enforces Redis-based rate limiting.	infra/nginx/nginx.conf; api-gateway/app/main.py; docker-compose.yml.
9. C4 diagrams (7)	Seven updated C4 diagrams were prepared for the new architecture.	Documentation/Block3-C4/images and Structurizr DSL workspace.
10. Infra part	Docker Compose local infrastructure is implemented. The report also explains the migration path to Kubernetes, Helm, GitOps, service mesh and load testing.	docker-compose.yml; Nginx, Kafka, Redis, PostgreSQL, Qdrant, Langfuse services.
11. RecSys	A separate recommender microservice was added with heuristic, content-based and educational collaborative filtering approaches.	recsys-service/app/main.py; frontend Smart recommendations block.

3. Requirement 5 - Kafka communication and EDA

Kafka is used as the central event broker. After each important business action, the service first writes its own data to its database and then publishes a domain event to Kafka. This makes the architecture event-driven: other future services can consume the same event stream without changing the original business service.

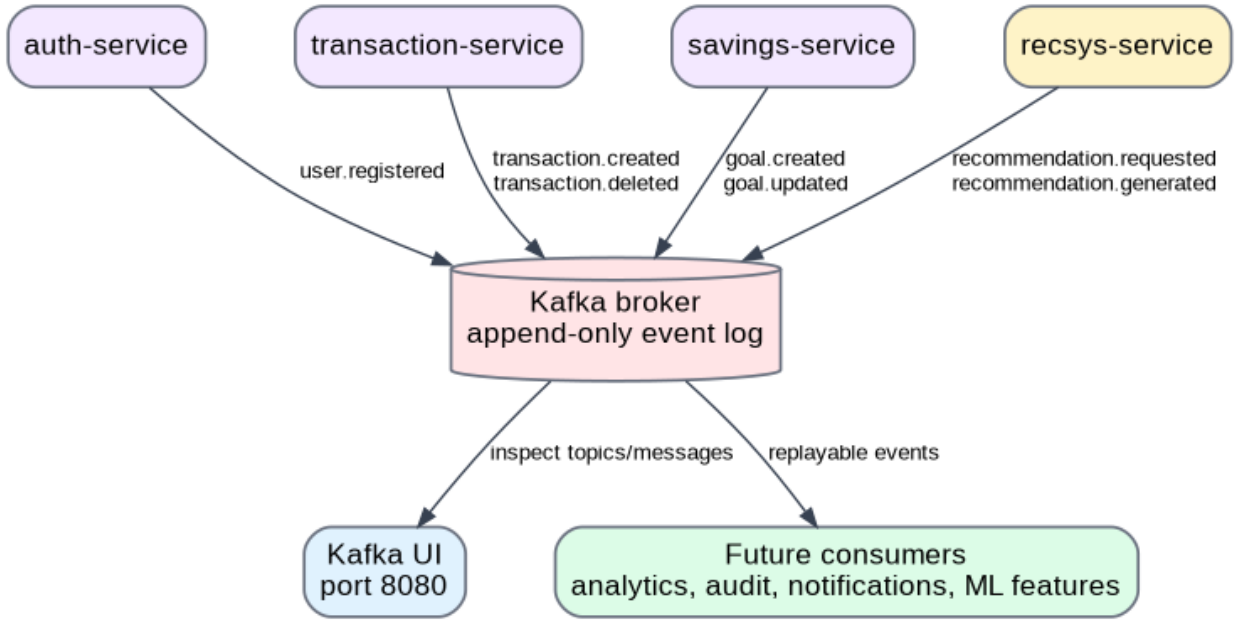

Figure 2. Kafka-based Event-Driven Architecture view.

Producer service	Event topics	Business meaning
auth-service	user.registered	A new user account was created.
transaction-service	transaction.created, transaction.deleted	A financial transaction was added or removed. The event can be used for analytics, audit, recommendations, or notifications.
savings-service	goal.created, goal.updated	A savings goal was created or its progress changed.
recsys-service	recommendation.requested, recommendation.generated	A recommendation request was received and recommendation output was produced.

3.1 Kafka vs RabbitMQ vs NATS

Criterion	Kafka	RabbitMQ	NATS
Main model	Distributed append-only event log with topics, partitions and offsets.	Message broker with queues, exchanges and acknowledgements.	Lightweight messaging system focused on speed and simplicity.
Best for	Event sourcing, replayable analytics, audit trails, high-throughput event streams.	Task queues, command delivery, request processing where every message is consumed once.	Low-latency service-to-service messaging and cloud-native lightweight communication.
Why / why not here	Selected because the project needs event history, Kafka UI, topic inspection and future analytics over financial events.	Good alternative for simple background jobs, but weaker for replayable event history.	Could be objectively better for a very small MVP where simplicity and low latency matter more than durable event log semantics.
Final decision	Chosen for this project.	Not selected.	Not selected.

Objectively, if the project only needed simple asynchronous commands, RabbitMQ would be easier. If the project only needed very lightweight low-latency communication, NATS would be simpler. For Kopilkin,

Kafka is the best fit because financial events are useful as a durable stream for analytics, audit, recommendation improvement and future consumers.

4. Requirement 6 - Data architecture

The project follows the database-per-service pattern at the logical database level. Each core service owns its own database and tables. This reduces coupling between services and prevents direct cross-service database access. Non-relational storage is used where relational tables are not the best match: Qdrant stores vector memory for the AI assistant and Redis stores short-lived cache/rate-limit data.

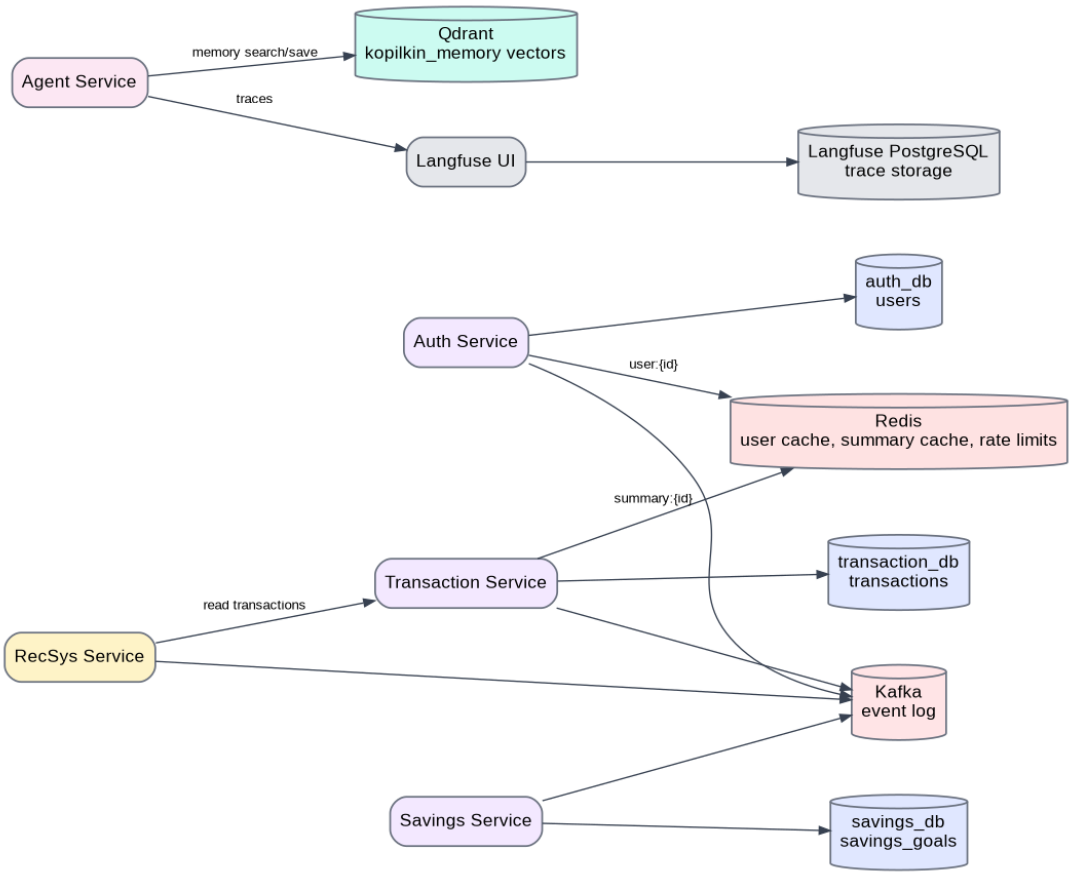

Figure 3. Data storage view.

Data type	Storage	Owner / user	Reason
Users	PostgreSQL auth_db, users table	auth-service	Structured user data requires uniqueness, indexing and transactions.
Transactions	PostgreSQL transaction_db, transactions table	transaction-service	Financial records are structured and require reliable persistence.
Savings goals	PostgreSQL savings_db, savings_goals table	savings-service	Goal state is structured and updated over time.
AI memory vectors	Qdrant collection kopilkin_memory	agent-service / Mem0	Semantic memory search is vector-based, not relational.
LLM traces	Langfuse PostgreSQL database	Langfuse	Trace storage is separate from business data.
Cache and rate limits	Redis	auth, transaction, gateway	Short-lived data with TTL and atomic counters.
Domain events	Kafka topics	all event producers	Append-only event log for EDA and future analytics.

5. Requirement 7 - Redis caching

Redis is used not only as a generic cache but also as part of the routing layer. The implementation contains three practical Redis use cases.

Use case	Key pattern	TTL / invalidation	Explanation
User profile cache	user:{user_id}	300 seconds; deleted after user update	auth-service avoids repeated PostgreSQL reads for profile data.
Transaction summary cache	summary:{user_id}	60 seconds; deleted after create/delete transaction	transaction-service avoids recalculating totals for every summary request.
Rate limiting	rate_limit:{client_ip}:{minute_window}	60 seconds	api-gateway increments Redis counters and returns HTTP 429 when the limit is exceeded.

```
Cache flow: request -> Redis GET -> cache hit returns immediately / cache miss reads PostgreSQL -> SETEX
with TTL -> response
Invalidation flow: write operation -> PostgreSQL update -> Kafka event -> Redis DEL old cache key
```

6. Requirement 8 - Routing layer: Router + Balancer + Rate Limiter

The frontend no longer needs to know every service port. It sends API requests to Nginx on port 8088. Nginx balances requests between two API Gateway instances. Each gateway applies rate limiting through Redis and then proxies the request to the correct internal microservice.

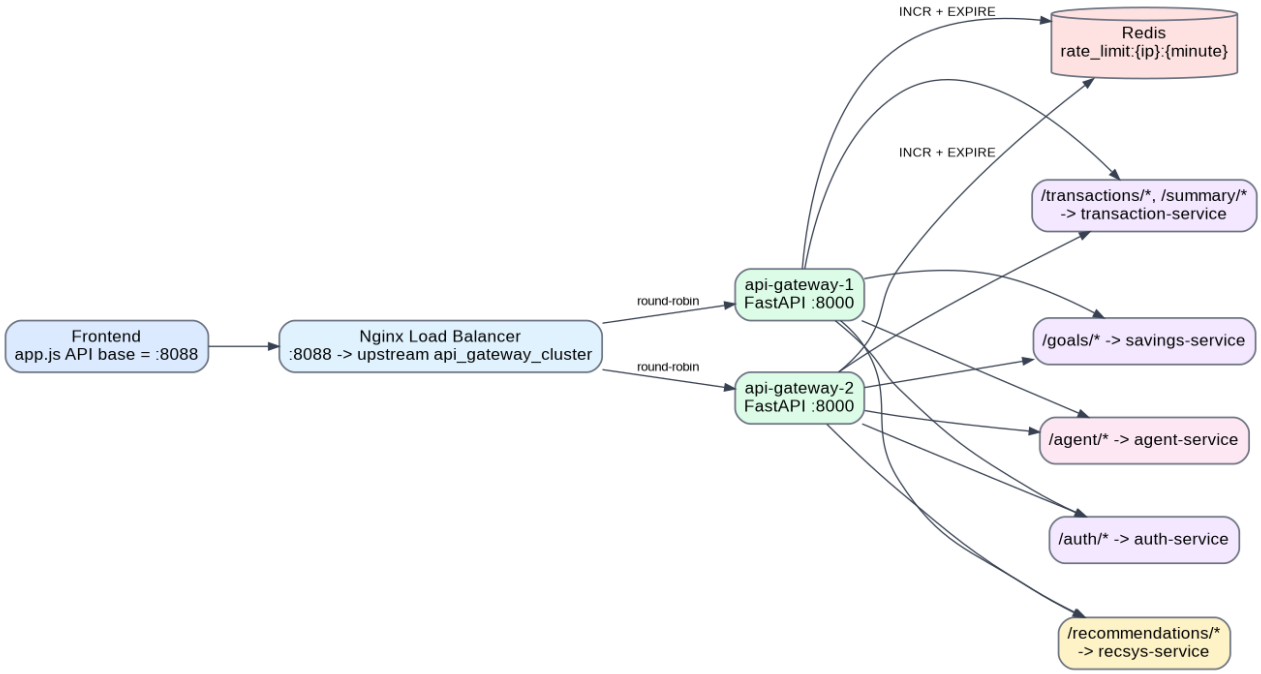

Figure 4. Routing layer with Nginx, API Gateway cluster and Redis rate limiter.

Route in API Gateway	Target service	Purpose
/auth/{path}	auth-service:8001	Registration, login, user profile.
/transactions and /transactions/{path}	transaction-service:8002	Create, list and delete transactions.
/summary/{user_id}	transaction-service:8002	Financial summary.
/goals and /goals/{path}	savings-service:8003	Savings goal management.
/agent/{path}	agent-service:8004	AI assistant chat and metrics.
/recommendations/{user_id}	recsys-service:8005	Smart recommendations.

7. Requirement 9 - Updated C4 diagrams

The architecture changed significantly in Task Block 3, so the previous C4 diagrams had to be updated. The new set contains seven diagrams: system context, container overview, data/events/routing view, gateway components, transaction service components, AI agent service components, and a dynamic add-transaction flow.

C4 L1 - System Context

Shows Kopilkin as a system used by a personal finance app user and connected to local LLM runtime, memory, observability and Kafka.

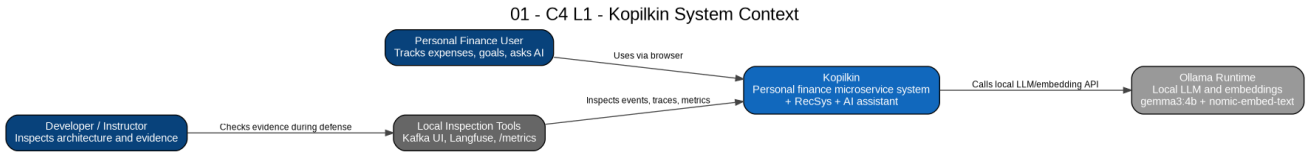

Figure - C4 L1 - System Context.

C4 L2 - Container Overview

Shows frontend, Nginx, API Gateway instances, business services, AI services, PostgreSQL, Redis, Kafka, Qdrant and Langfuse.

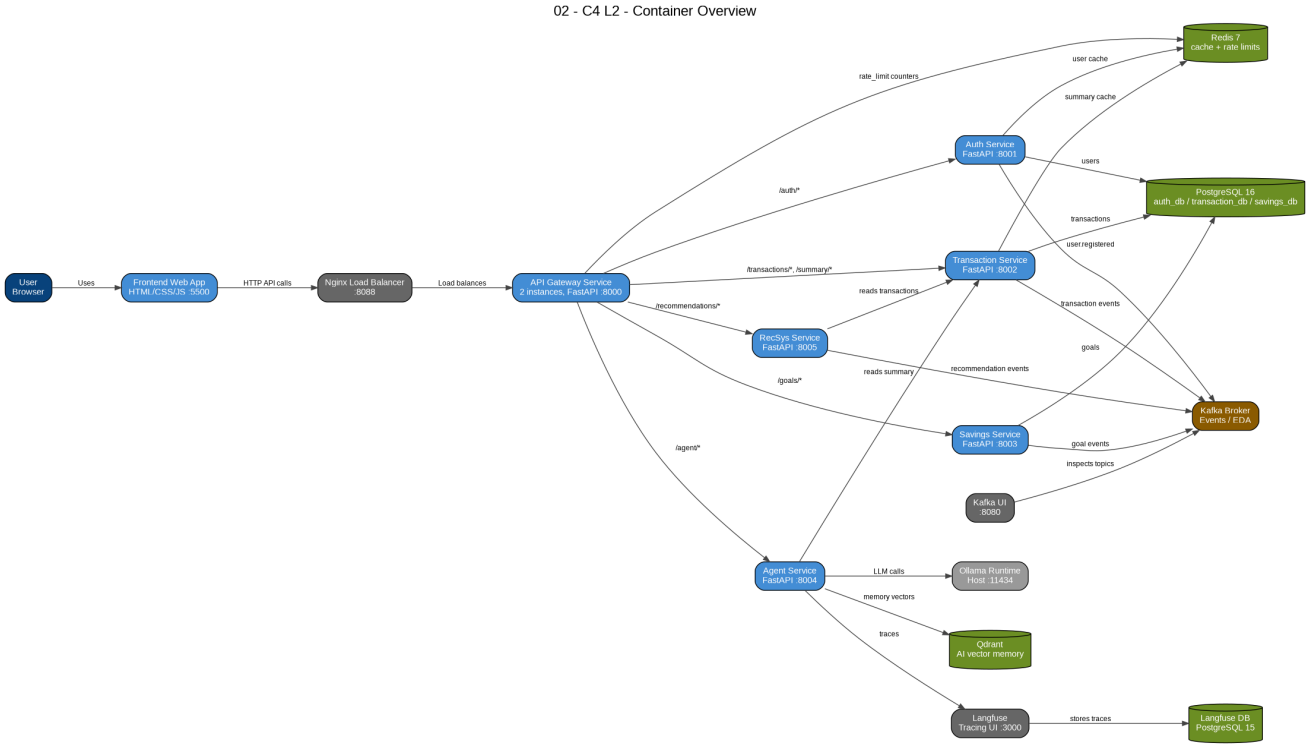

Figure - C4 L2 - Container Overview.

C4 L2 - Data, Events and Routing View

Focuses on the relationship between routing, service-owned data, Kafka topics and caching.

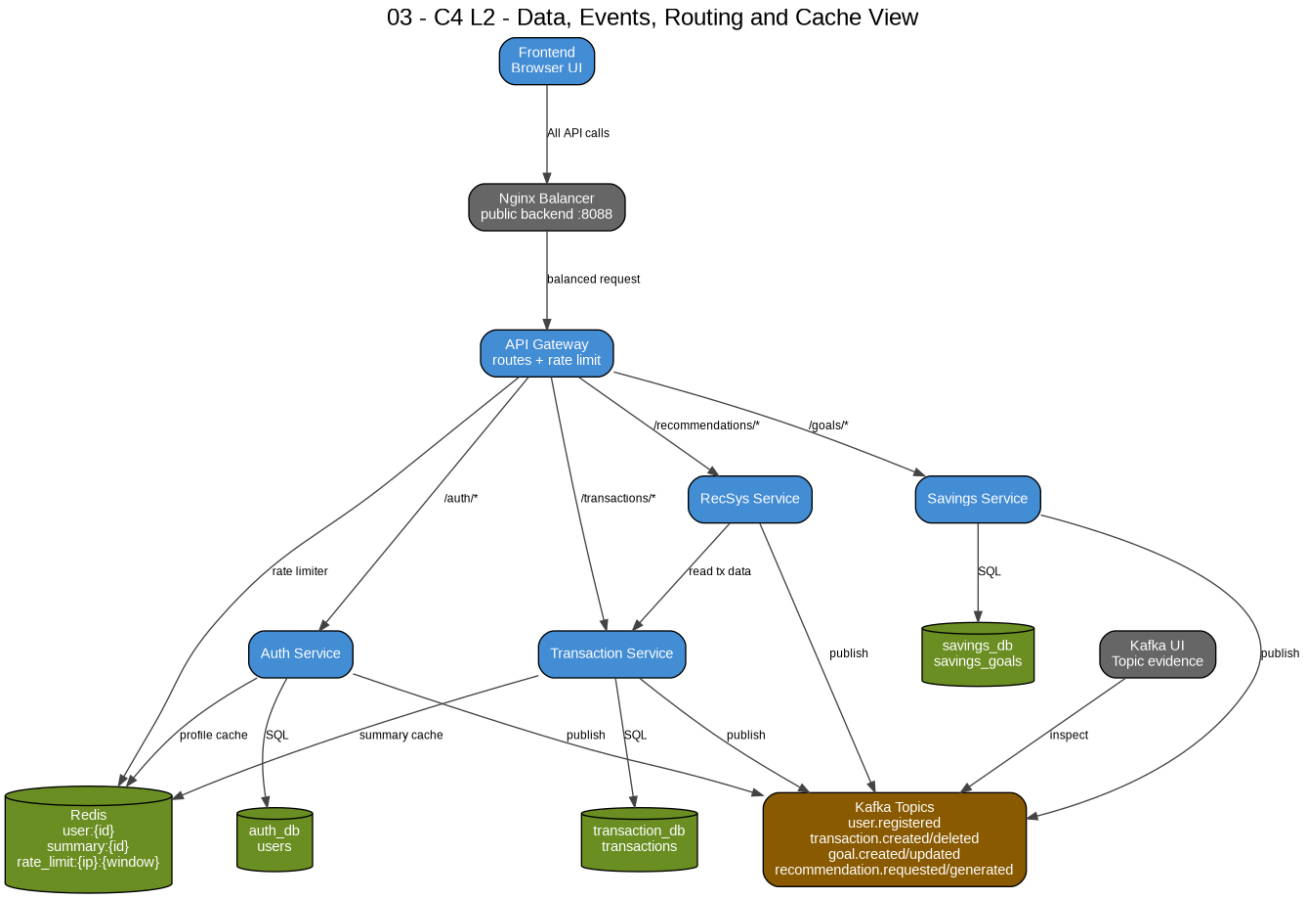

Figure - C4 L2 - Data, Events and Routing View.

C4 L3 - API Gateway Components

Explains the internal gateway structure: CORS, rate limit middleware, proxy logic, route handlers and service URL configuration.

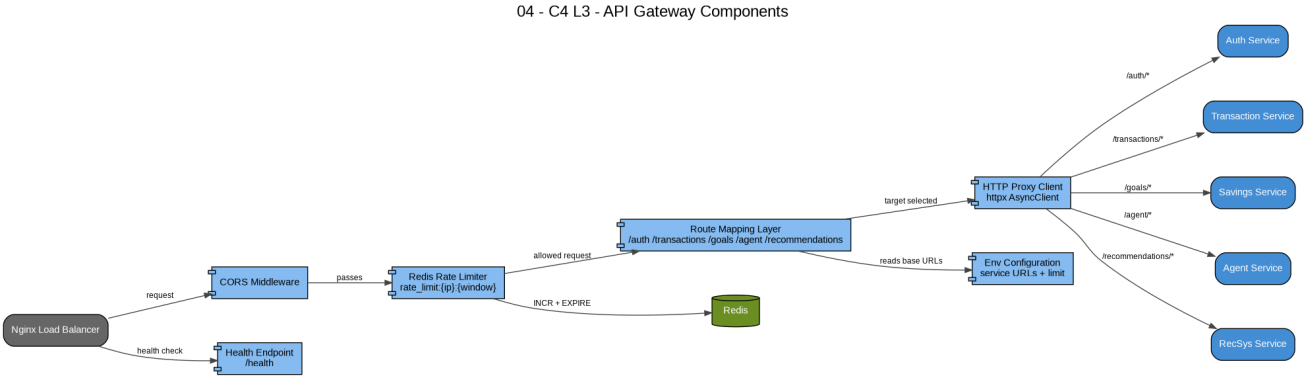

Figure - C4 L3 - API Gateway Components.

C4 L3 - Transaction Service Components

Shows transaction endpoints, SQLAlchemy persistence, Redis summary cache and Kafka event publishing.

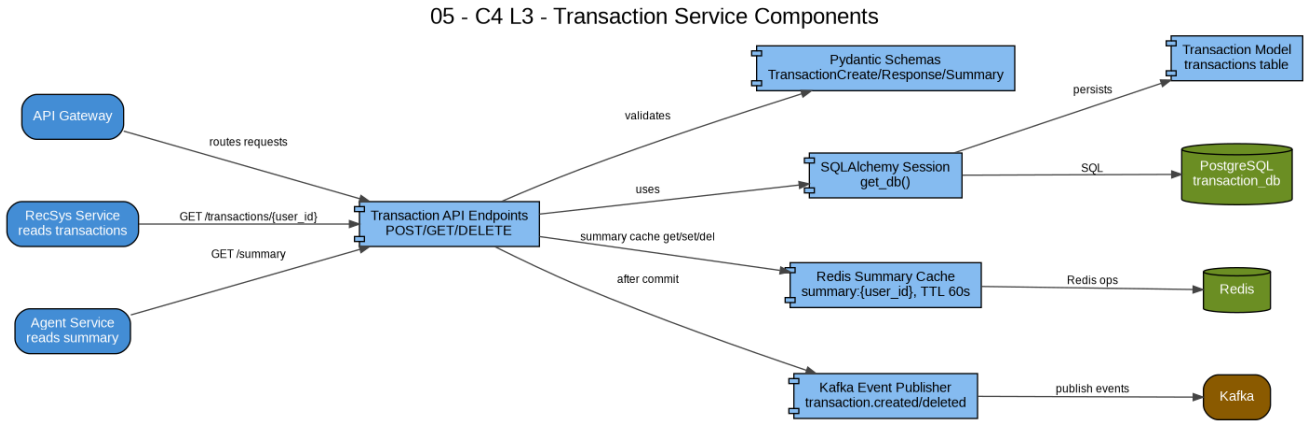

Figure - C4 L3 - Transaction Service Components.

C4 L3 - AI Agent Service Components

Shows the multi-agent flow with orchestrator, memory, router, analyst, advisor, LLM, Qdrant and Langfuse.

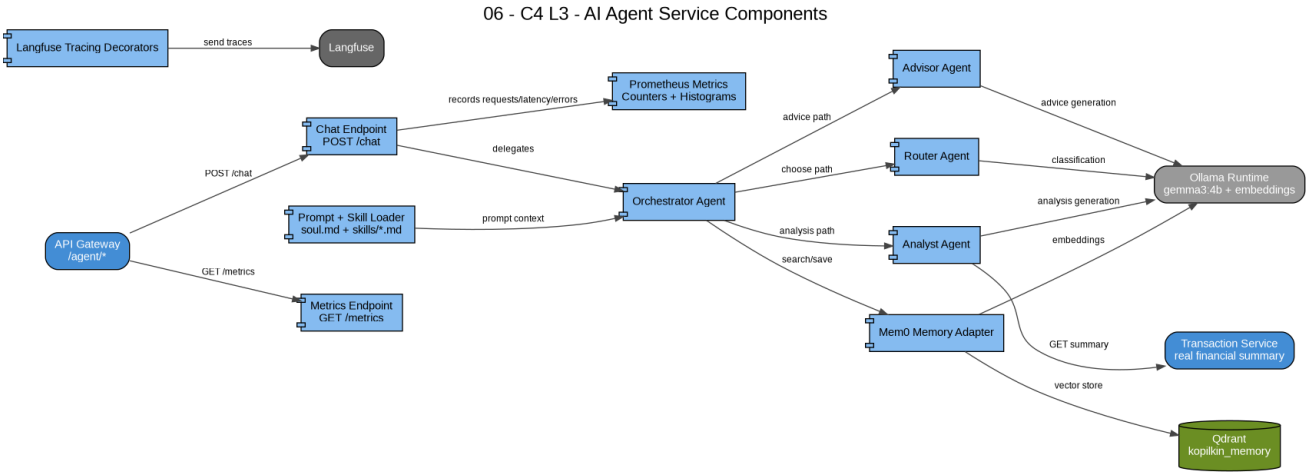

Figure - C4 L3 - AI Agent Service Components.

C4 Dynamic - Add Transaction Flow

Shows the runtime sequence when a user creates a transaction through the new routing layer.

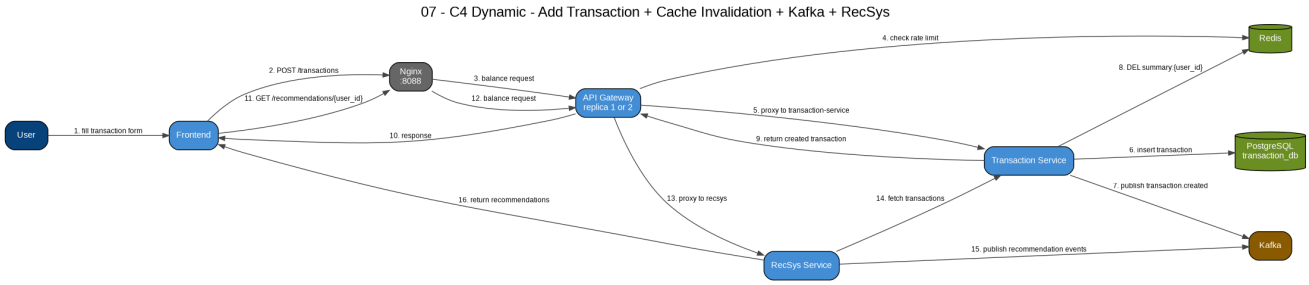

Figure - C4 Dynamic - Add Transaction Flow.

8. Requirement 10 - Infrastructure part

For the current student MVP, the infrastructure is implemented with Docker Compose. This is practical because it gives reproducible service isolation without the operational overhead of a full Kubernetes cluster. The project already contains real infrastructure components: PostgreSQL, Kafka, Kafka UI, Redis, Qdrant, Langfuse, Langfuse PostgreSQL, Nginx and multiple API Gateway instances.

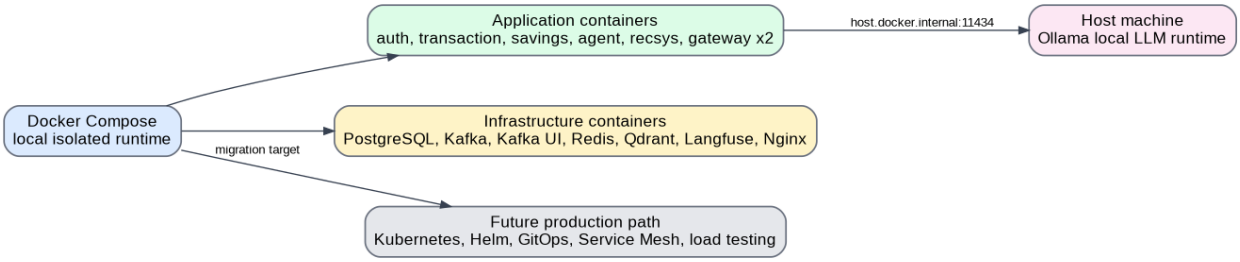

Figure 5. Current local infrastructure and future production direction.

Infrastructure area	Current implementation	Production / future improvement
Runtime isolation	Docker Compose containers for services and infrastructure.	Kubernetes cluster such as k3s/k0s/Talos can replace Compose for production-like deployment.
Traffic entry point	Nginx load balancer on port 8088.	Ingress Controller, HAProxy/Keepalived or service mesh ingress.
Scalability	Two API Gateway instances behind Nginx.	Horizontal Pod Autoscaler and autoscaling policies.
Caching / rate limiting	Redis container.	Managed Redis/Valkey cluster, stronger distributed rate limiting rules.
Event broker	Single Kafka broker for local development.	Multi-broker Kafka cluster with replication, monitoring and retention policies.
Observability	Langfuse traces, /metrics in agent-service, structured logs.	Grafana dashboards, Prometheus scraping, alerting, trace correlation and log aggregation.

```
Main local run command:
docker compose up --build

Useful checks:
curl http://127.0.0.1:8088/health
open http://127.0.0.1:8080 for Kafka UI
open http://127.0.0.1:3000 for Langfuse
```

9. Requirement 11 - Recommender System

The recommender system is implemented as a separate FastAPI microservice. It is not hard-coded only on the frontend: it reads the user transactions from transaction-service, generates recommendations and publishes recommendation events to Kafka.

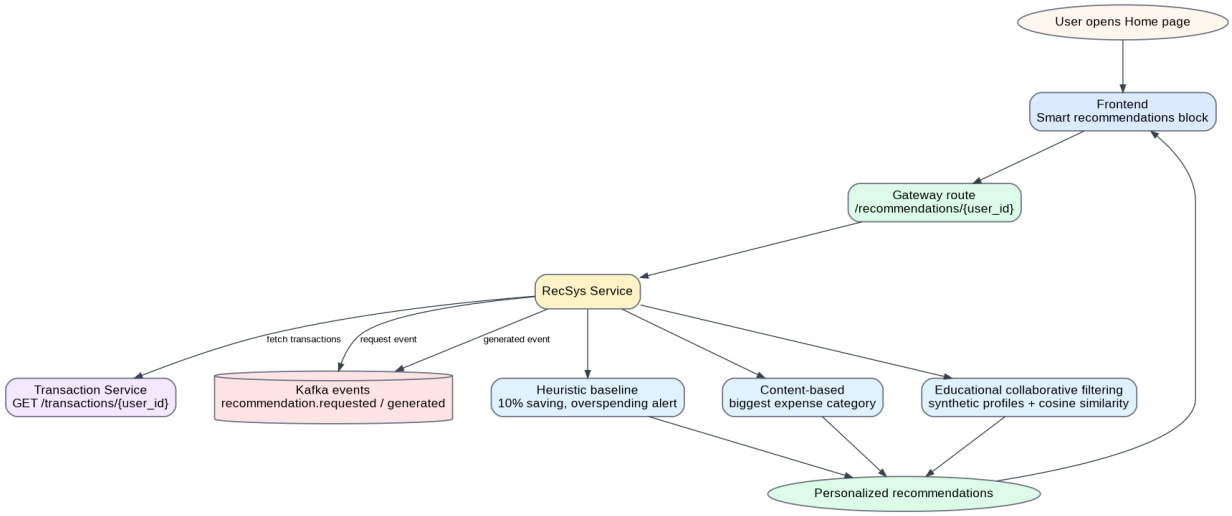

Figure 6. RecSys flow.

Approach	Implementation in recsys-service	Example output
Heuristic baseline	Uses simple financial rules such as saving 10% of income and detecting expenses higher than	Save 10% of your income; overspending alert.

	income.	
Content-based recommendation	Finds the largest expense category and generates advice based on that category.	Your biggest spending category is Travel. Create a separate travel savings goal.
Educational collaborative filtering	Builds a category vector and compares it with synthetic user profiles using cosine similarity.	Recommendation based on similar users: traveler / shopper / student_saver.
Cold start	If the user has no transactions, it suggests adding transactions and creating the first savings goal.	Start tracking your first expenses.

10. Conclusion

Task Block 3 significantly improved Kopilkin. The project now has a clearer microservice architecture, persistent service-owned data, Kafka-based event publishing, Redis caching and rate limiting, an API Gateway with Nginx load balancing, a recommender microservice, and a complete updated set of seven C4 diagrams.

Блок заданий 3 - Микросервисы, данные, маршрутизация, инфраструктура и рекомендательная система

Kopilkin Personal Finance Application

Mubarakov Islam

1. Обзор проекта

Kopilkin - это mobile-first веб-приложение для управления личными финансами. Пользователь может зарегистрироваться, войти в аккаунт, добавлять доходы и расходы, создавать цели накоплений, задавать вопросы AI-ассистенту о своих тратах и получать умные финансовые рекомендации. В рамках третьего блока заданий проект был расширен от простого MVP до микросервисной системы с Kafka-событиями, PostgreSQL-базами данных, Redis-кэшированием, API Gateway, Nginx-балансировщиком и отдельным сервисом рекомендаций.

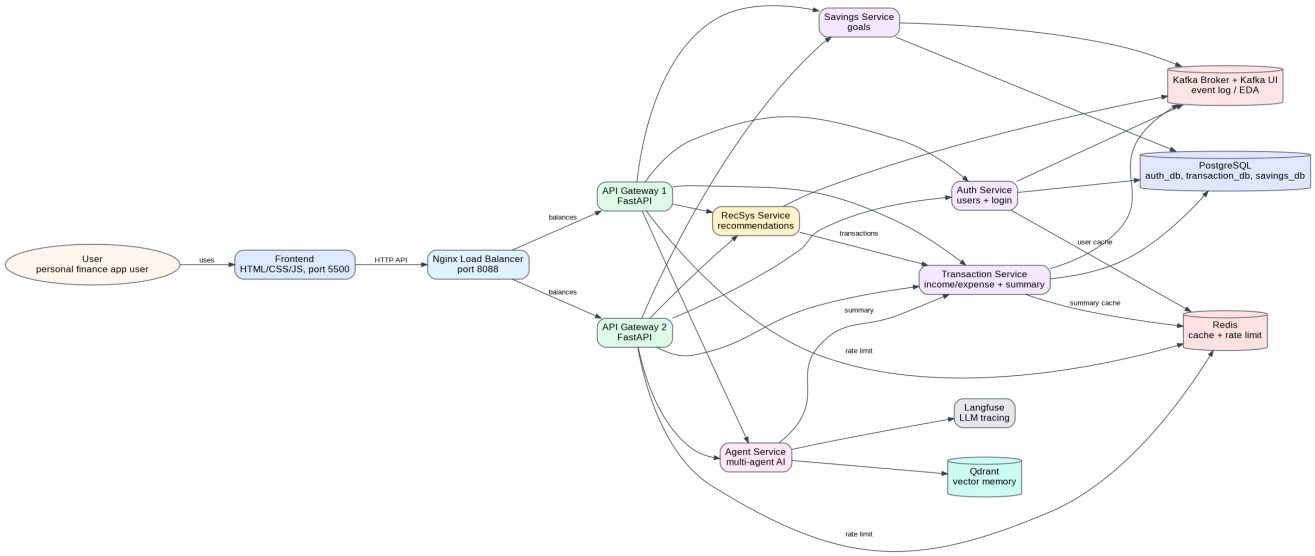

Рисунок 1. Обновлённая архитектура Block 3.

2. Объём третьего блока заданий

Этот отчёт описывает весь третий блок заданий. Ниже показано, как требования 5-11 связаны с реальной реализацией проекта.

Требование	Результат реализации	Где видно в проекте
5. Communication via Kafka / EDA	Добавлены Kafka broker, Kafka UI и producer-логика. Сервисы публикуют доменные события после бизнес-операций.	docker-compose.yml; app/events.py в auth, transaction, savings и recsys сервисах.
6. Data: RDBMS + NoSQL / cold storage	Основные бизнес-данные перенесены в PostgreSQL. Qdrant хранит vector memory. Langfuse использует отдельную PostgreSQL БД. Kafka хранит события, Redis - временные данные.	infra/postgres/init.sql; database.py и models.py в сервисах; контейнеры qdrant/langfuse.
7. Redis-like caching	Redis используется для кэша профиля пользователя, кэша summary транзакций и rate limiting counters.	auth-service/app/cache.py; transaction-service/app/cache.py; api-gateway/app/main.py.
8. Routing layer	Nginx балансирует трафик между двумя API Gateway. Gateway маршрутизирует запросы во внутренние сервисы и ограничивает частоту запросов через Redis.	infra/nginx/nginx.conf; api-gateway/app/main.py; docker-compose.yml.
9. C4 diagrams (7)	Подготовлены семь обновлённых C4-диаграмм под новую архитектуру.	Documentation/Block3-C4/images и Structurizr DSL workspace.
10. Infra part	Реализована локальная Docker Compose инфраструктура. Также описан путь развития к Kubernetes, Helm, GitOps, service mesh и load testing.	docker-compose.yml; Nginx, Kafka, Redis, PostgreSQL, Qdrant, Langfuse.
11. RecSys	Добавлен отдельный recommender microservice с heuristic, content-based и учебным collaborative filtering подходом.	recsys-service/app/main.py; frontend Smart recommendations block.

3. Требование 5 - Kafka communication и EDA

Kafka используется как центральный брокер событий. После важного бизнес-действия сервис сначала сохраняет свои данные в собственную базу, а затем публикует доменное событие в Kafka. Благодаря этому архитектура становится event-driven: будущие сервисы смогут читать тот же поток событий без изменения исходных бизнес-сервисов.

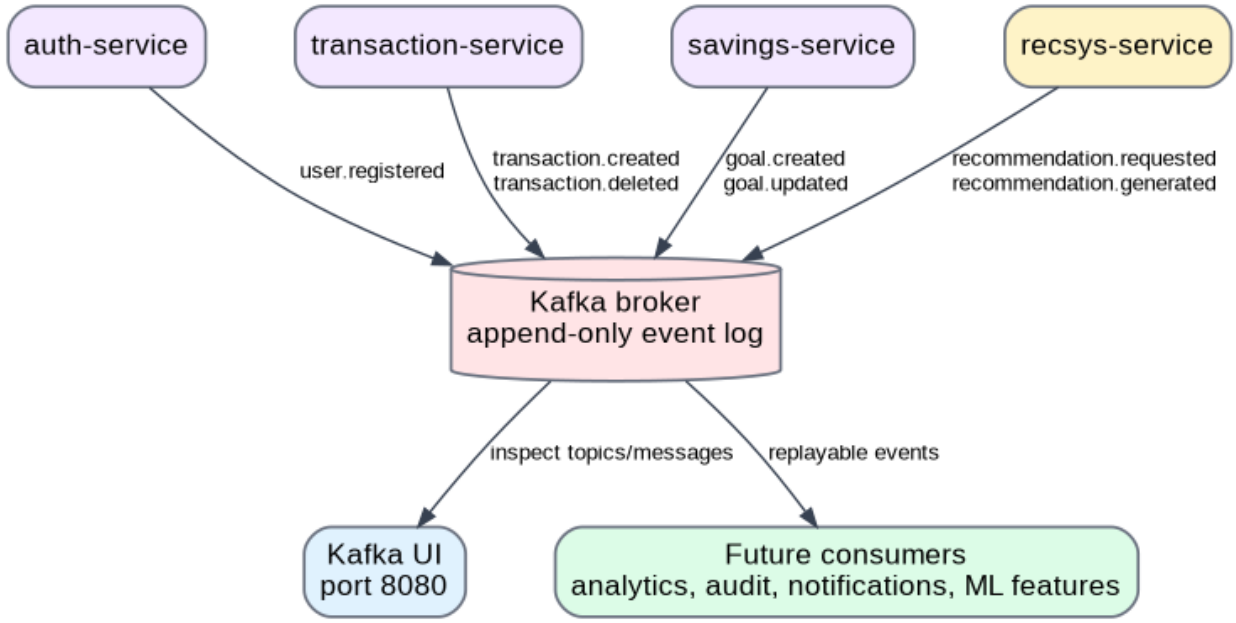

Рисунок 2. Event-Driven Architecture на Kafka.

Producer service	Kafka topics	Смысл события
auth-service	user.registered	Создан новый пользователь.
transaction-service	transaction.created, transaction.deleted	Финансовая транзакция добавлена или удалена. Эти события подходят для аналитики, аудита, рекомендаций и уведомлений.
savings-service	goal.created, goal.updated	Создана цель накоплений или изменён прогресс цели.
recsys-service	recommendation.requested, recommendation.generated	Получен запрос на рекомендации и сформирован результат рекомендаций.

3.1 Сравнение Kafka, RabbitMQ и NATS

Критерий	Kafka	RabbitMQ	NATS
Основная модель	Распределённый append-only event log с topics, partitions и offsets.	Брокер сообщений с queues, exchanges и acknowledgements.	Лёгкая система сообщений с фокусом на скорость и простоту.
Для чего лучше подходит	Event sourcing, replayable analytics, audit trail, потоковые события с высокой нагрузкой.	Task queues, command delivery, обработка задач, где сообщение должно быть обработано конкретным consumer-ом.	Очень быстрые service-to-service сообщения и простая cloud-native коммуникация.
Почему / почему не здесь	Выбран, потому что проекту полезны история событий, Kafka UI, просмотр topics и будущая аналитика финансовых событий.	Хорош для простых фоновых задач, но слабее подходит для replayable event history.	Мог бы быть объективно лучше для маленького MVP, где важнее простота и низкая задержка, а не durable event log.
Решение	Выбран для этого проекта.	Не выбран.	Не выбран.

Объективно, если бы проекту нужны были только простые асинхронные команды, RabbitMQ был бы проще. Если бы нужна была только максимально лёгкая low-latency коммуникация, NATS был бы проще. Для Kopilkin Kafka подходит лучше, потому что финансовые события полезны как долговременный поток для аналитики, аудита, развития рекомендаций и будущих consumers.

4. Требование 6 - Архитектура данных

Проект использует database-per-service pattern на уровне логических баз данных. Каждый core service владеет своей базой и своими таблицами. Это снижает связанность сервисов и не заставляет один сервис напрямую читать таблицы другого. Нереляционные хранилища используются там, где таблицы не являются лучшим решением: Qdrant хранит vector memory для AI-ассистента, а Redis хранит временный кэш и rate limit counters.

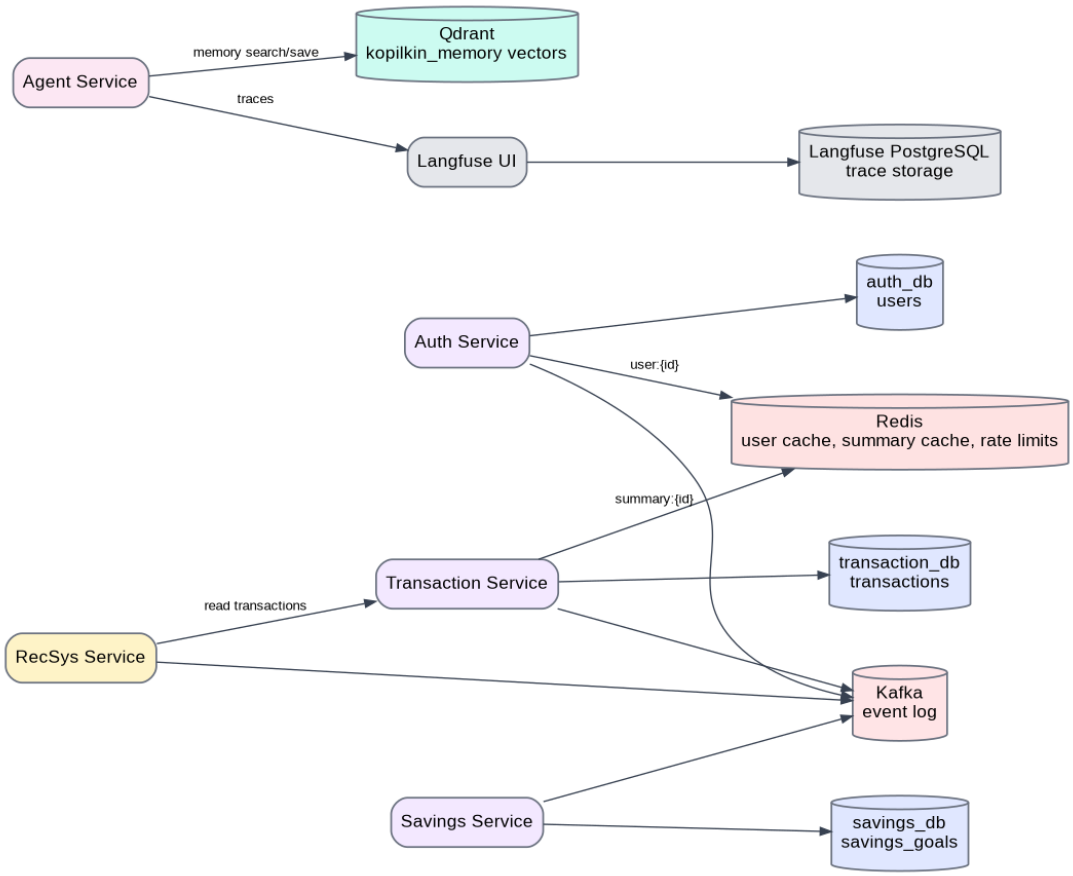

Рисунок 3. Хранилища данных в проекте.

Тип данных	Хранилище	Владелец / пользователь	Причина
Пользователи	PostgreSQL auth_db, таблица users	auth-service	Структурированные данные пользователя требуют unique constraints, индексов и транзакций.
Транзакции	PostgreSQL transaction_db, таблица transactions	transaction-service	Финансовые записи структурированы и должны храниться надёжно.
Цели накоплений	PostgreSQL savings_db, таблица savings_goals	savings-service	Состояние цели структурировано и меняется со временем.
AI memory vectors	Qdrant collection kopilkin_memory	agent-service / Mem0	Semantic memory search работает через векторы, а не через обычные таблицы.
LLM traces	Langfuse PostgreSQL database	Langfuse	Трейсы вынесены отдельно от бизнес-данных.
Cache и rate limits	Redis	auth, transaction, gateway	Временные данные с TTL и атомарными счётчиками.
Domain events	Kafka topics	event producers	Append-only event log для EDA и будущей аналитики.

5. Требование 7 - Redis caching

Redis используется не просто как абстрактный кэш, а в трёх практических местах архитектуры.

Use case	Key pattern	TTL / invalidation	Объяснение
Кэш профиля пользователя	user:{user_id}	300 секунд; удаляется после обновления пользователя	auth-service не делает повторные чтения PostgreSQL для профиля.
Кэш summary транзакций	summary:{user_id}	60 секунд; удаляется после create/delete transaction	transaction-service не пересчитывает total income/expense на каждый запрос.
Rate limiting	rate_limit:{client_ip}:{minute_window}	60 секунд	api-gateway увеличивает Redis counters и возвращает HTTP 429 при превышении лимита.

```
Cache flow: request -> Redis GET -> cache hit returns immediately / cache miss reads PostgreSQL -> SETEX
with TTL -> response
Invalidation flow: write operation -> PostgreSQL update -> Kafka event -> Redis DEL old cache key
```

6. Требование 8 - Routing layer: Router + Balancer + Rate Limiter

Frontend больше не должен знать порты всех сервисов. Он отправляет API-запросы на Nginx по порту 8088. Nginx балансирует запросы между двумя API Gateway instances. Каждый gateway применяет Redis-based rate limiting и затем проксирует запрос в нужный внутренний микросервис.

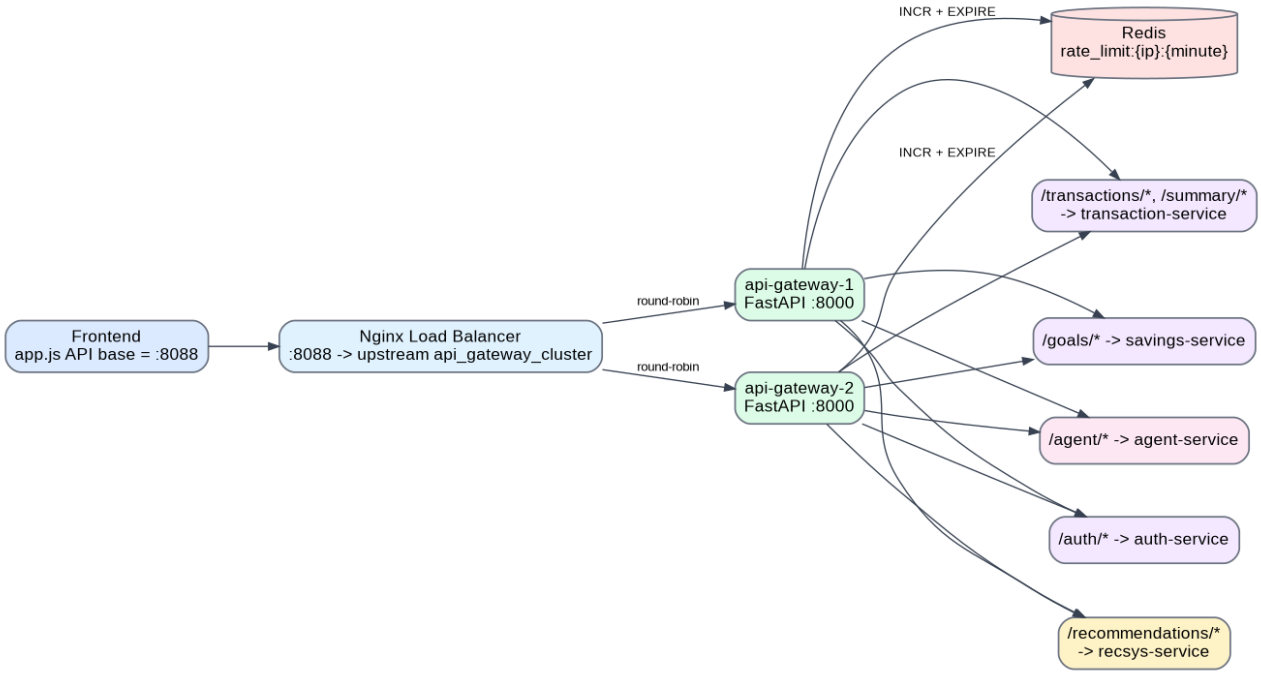

Рисунок 4. Routing layer с Nginx, API Gateway cluster и Redis rate limiter.

Route in API Gateway	Target service	Назначение
/auth/{path}	auth-service:8001	Регистрация, логин, профиль пользователя.
/transactions and /transactions/{path}	transaction-service:8002	Создание, просмотр и удаление транзакций.
/summary/{user_id}	transaction-service:8002	Финансовое summary пользователя.
/goals and /goals/{path}	savings-service:8003	Управление целями накоплений.
/agent/{path}	agent-service:8004	AI assistant chat и metrics.
/recommendations/{user_id}	recsys-service:8005	Smart recommendations.

7. Требование 9 - Обновлённые C4-диаграммы

Архитектура в Task Block 3 заметно изменилась, поэтому старые C4-диаграммы нужно было обновить. Новый набор состоит из семи диаграмм: system context, container overview, data/events/routing view, gateway components, transaction service components, AI agent service components и dynamic add-transaction flow.

C4 L1 - System Context

Показывает Kopilkin как систему, которой пользуется пользователь personal finance app, а также связи с local LLM runtime, memory, observability и Kafka.

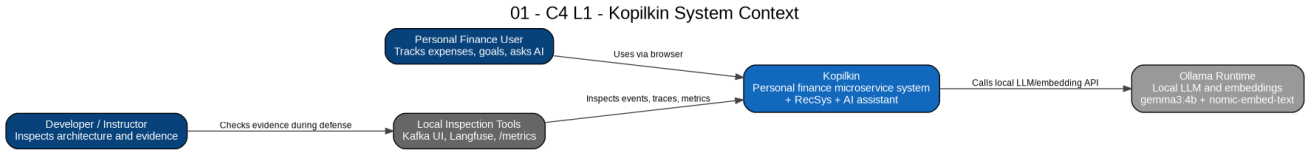

Рисунок - C4 L1 - System Context.

C4 L2 - Container Overview

Показывает frontend, Nginx, API Gateway instances, business services, AI services, PostgreSQL, Redis, Kafka, Qdrant и Langfuse.

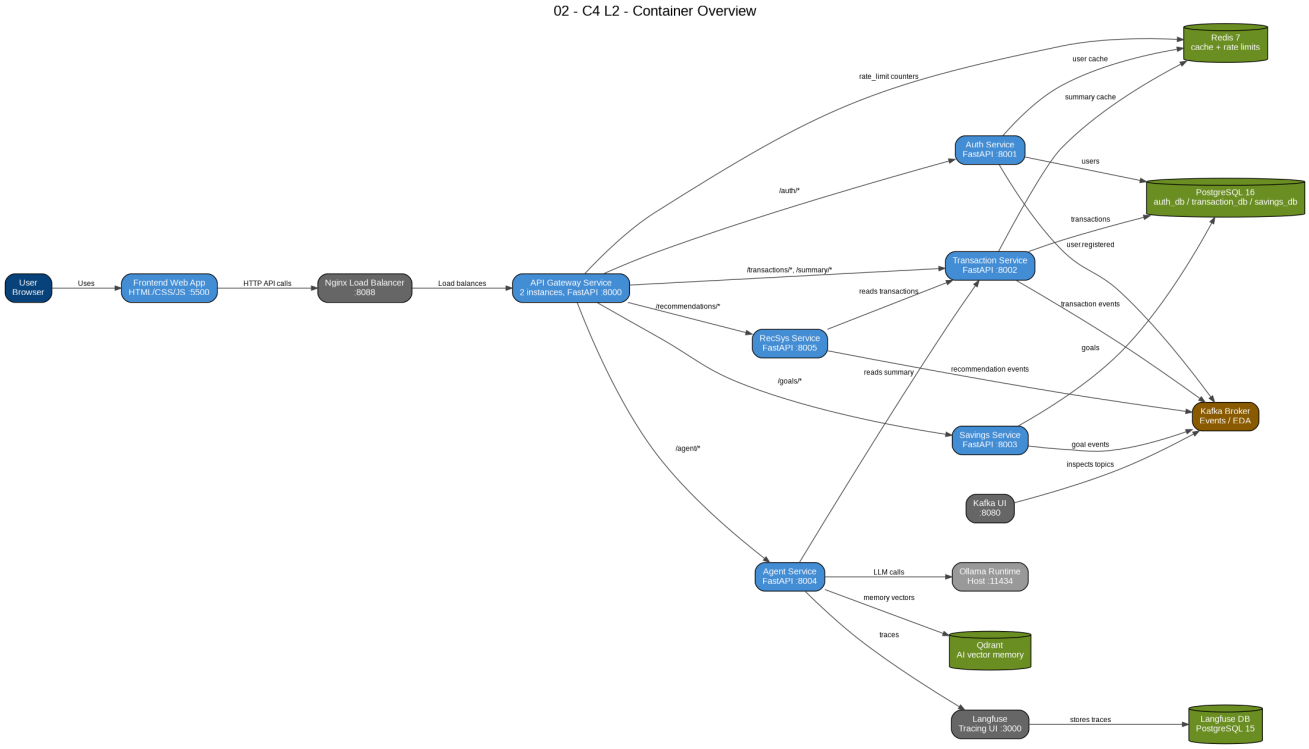

Рисунок - C4 L2 - Container Overview.

C4 L2 - Data, Events and Routing View

Фокусируется на связях между routing, service-owned data, Kafka topics и caching.

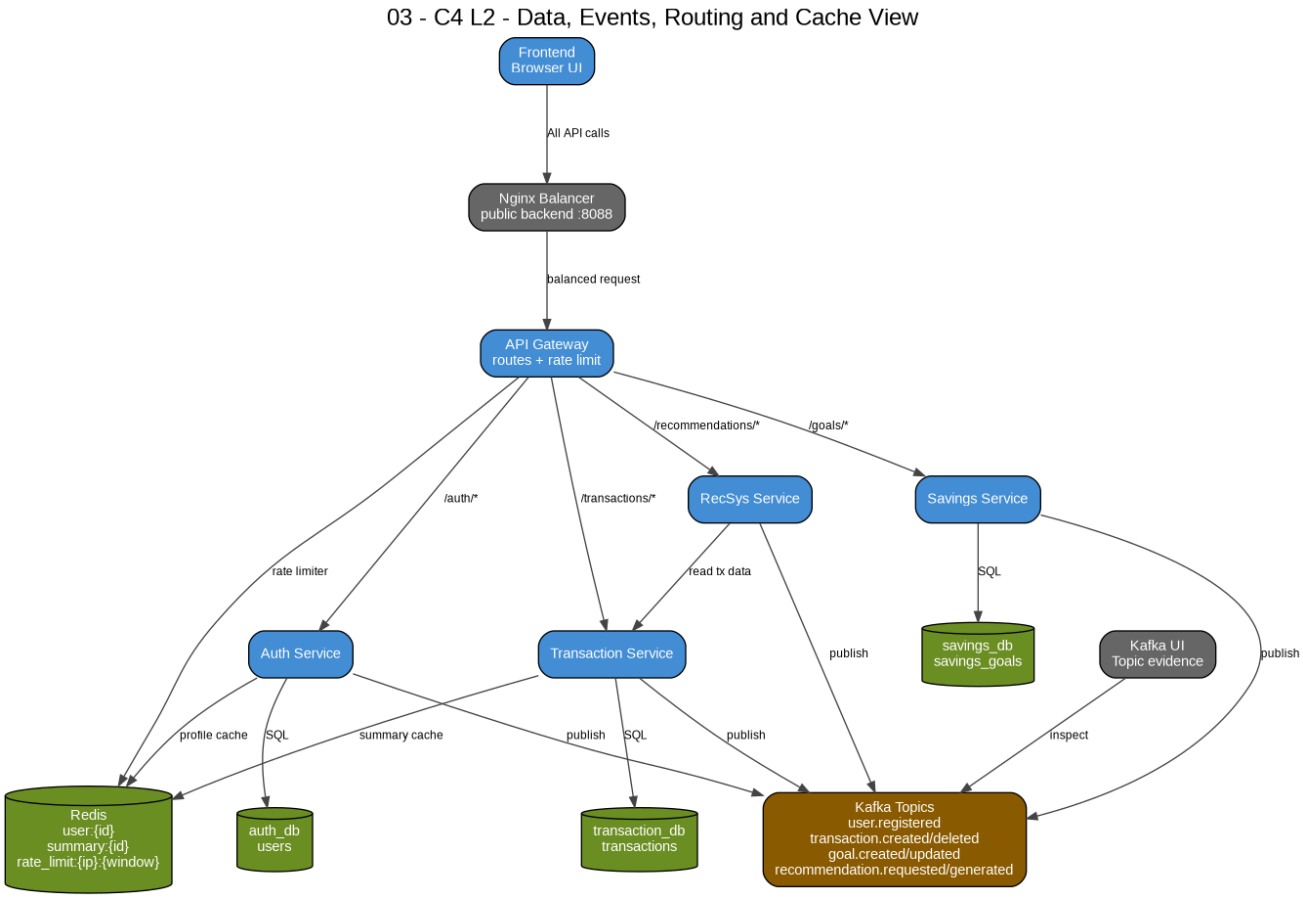

Рисунок - C4 L2 - Data, Events and Routing View.

C4 L3 - API Gateway Components

Объясняет внутреннюю структуру gateway: CORS, rate limit middleware, proxy logic, route handlers и service URL configuration.

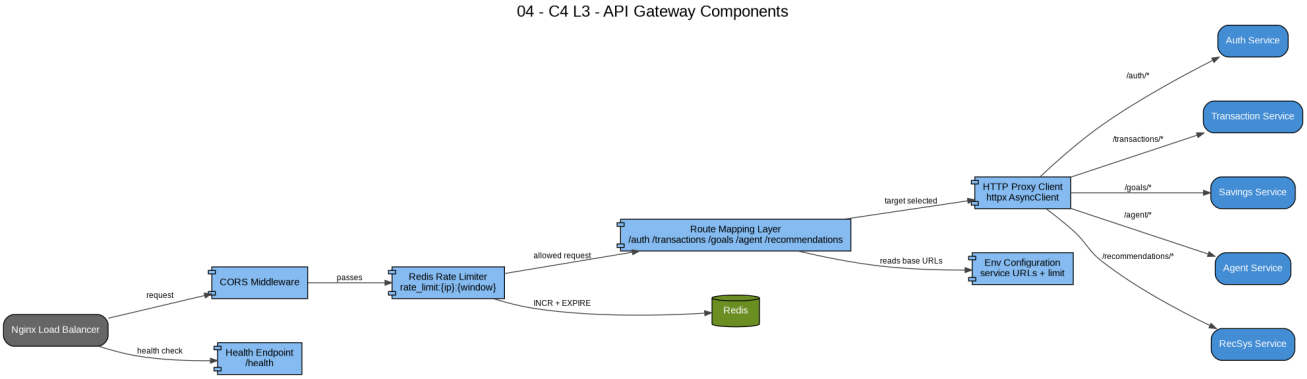

Рисунок - C4 L3 - API Gateway Components.

C4 L3 - Transaction Service Components

Показывает transaction endpoints, SQLAlchemy persistence, Redis summary cache и Kafka event publishing.

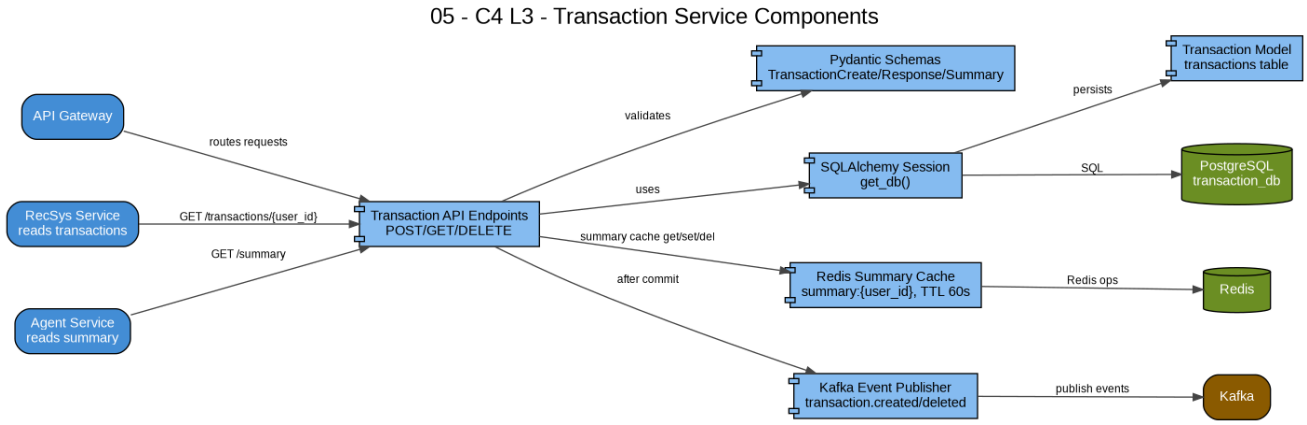

Рисунок - C4 L3 - Transaction Service Components.

C4 L3 - AI Agent Service Components

Показывает multi-agent flow: orchestrator, memory, router, analyst, advisor, LLM, Qdrant и Langfuse.

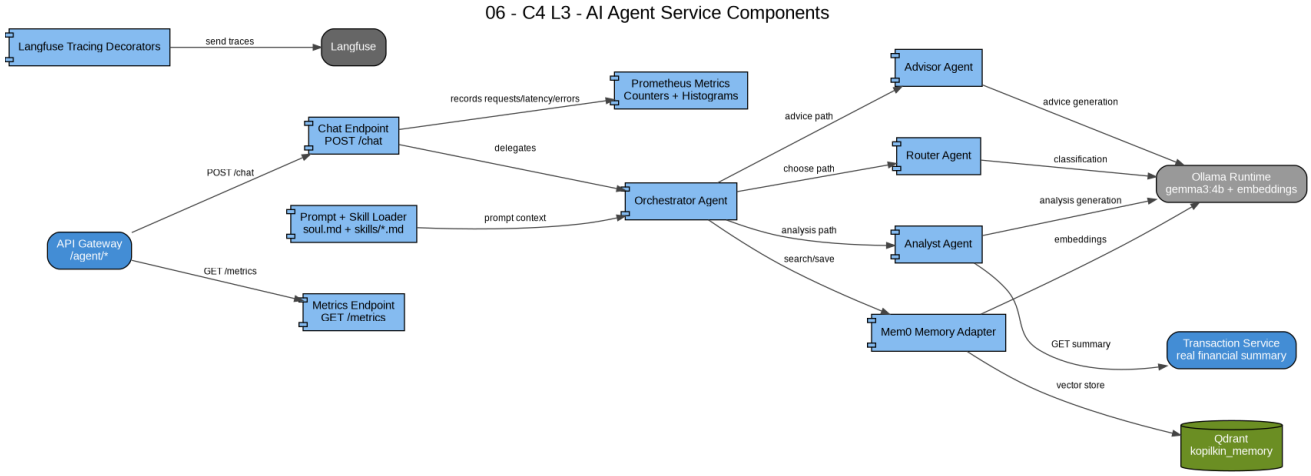

Рисунок - C4 L3 - AI Agent Service Components.

C4 Dynamic - Add Transaction Flow

Показывает runtime sequence, когда пользователь создаёт transaction через новый routing layer.

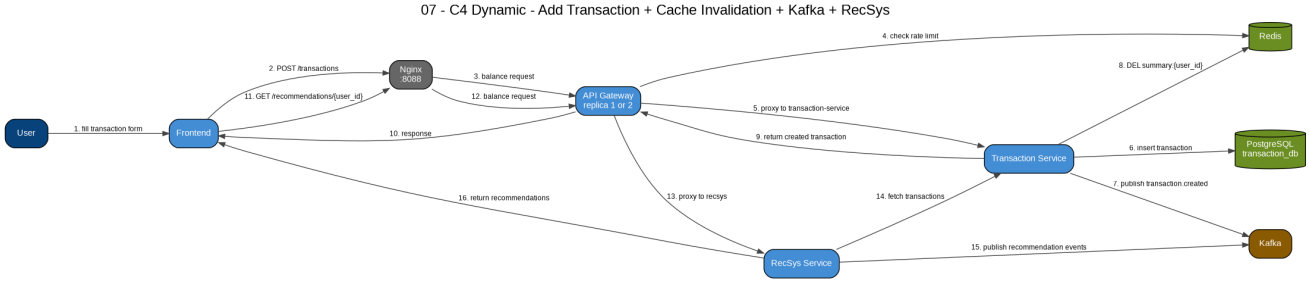

Рисунок - C4 Dynamic - Add Transaction Flow.

8. Требование 10 - Infrastructure part

Для текущего student MVP инфраструктура реализована через Docker Compose. Это практично, потому что даёт воспроизводимую изоляцию сервисов без слишком большого overhead полного Kubernetes-кластера. В проекте уже есть реальные infrastructure components: PostgreSQL, Kafka, Kafka UI, Redis, Qdrant, Langfuse, Langfuse PostgreSQL, Nginx и несколько API Gateway instances.

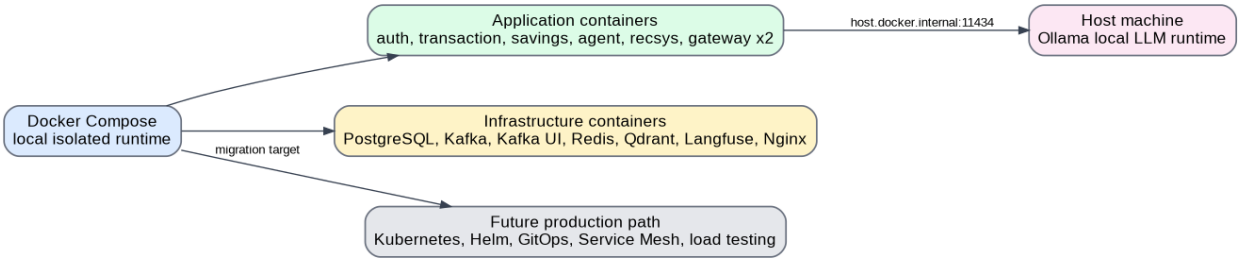

Рисунок 5. Текущая локальная инфраструктура и направление развития.

Infrastructure area	Текущая реализация	Production / future improvement
Runtime isolation	Docker Compose containers для сервисов и инфраструктуры.	Kubernetes cluster, например k3s/k0s/Talos, может заменить Compose для production-like deployment.
Traffic entry point	Nginx load balancer на порту 8088.	Ingress Controller, HAProxy/Keepalived или service mesh ingress.
Scalability	Два API Gateway instances за Nginx.	Horizontal Pod Autoscaler и autoscaling policies.
Caching / rate limiting	Redis container.	Managed Redis/Valkey cluster и более строгие distributed rate limiting rules.
Event broker	Один Kafka broker для local development.	Multi-broker Kafka cluster с replication, monitoring и retention policies.
Observability	Langfuse traces, /metrics в agent-service, structured logs.	Grafana dashboards, Prometheus scraping, alerting, trace correlation и log aggregation.

```
Main local run command:
docker compose up --build

Useful checks:
curl http://127.0.0.1:8088/health
open http://127.0.0.1:8080 for Kafka UI
open http://127.0.0.1:3000 for Langfuse
```

9. Требование 11 - Recommender System

Recommender system реализован как отдельный FastAPI microservice. Это не просто hard-coded текст на frontend: сервис читает транзакции пользователя из transaction-service, генерирует рекомендации и публикует recommendation events в Kafka.

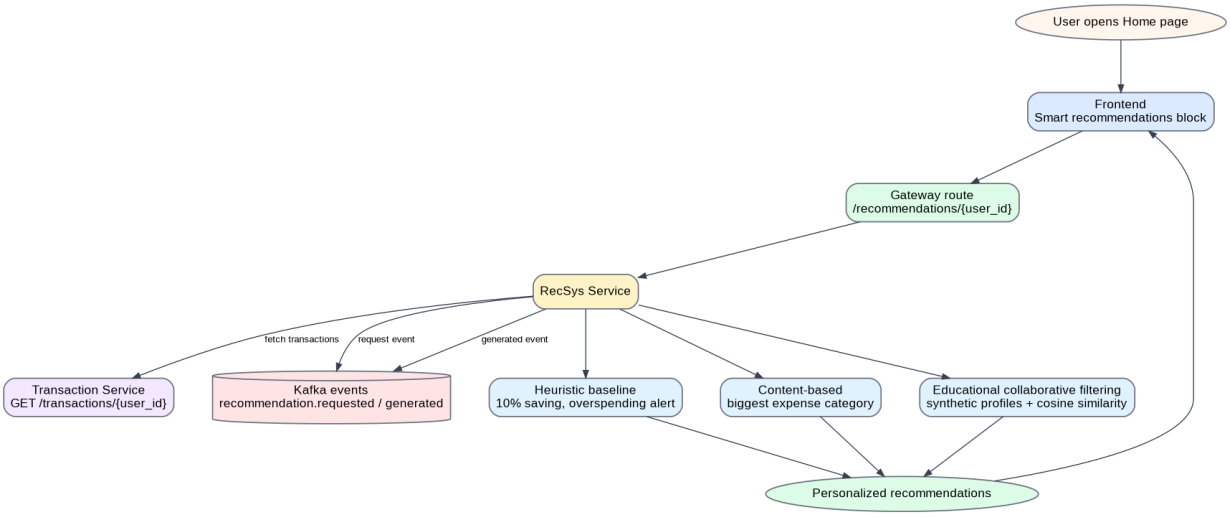

Рисунок 6. RecSys flow.

Approach	Реализация в recsys-service	Пример результата
Heuristic baseline	Использует простые финансовые правила: откладывать 10% дохода и замечать ситуацию, когда расходы выше доходов.	Save 10% of your income; overspending alert.

Content-based recommendation	Находит самую большую expense category и даёт совет по этой категории.	Your biggest spending category is Travel. Create a separate travel savings goal.
Educational collaborative filtering	Строит category vector и сравнивает его с synthetic user profiles через cosine similarity.	Recommendation based on similar users: traveler / shopper / student_saver.
Cold start	Если у пользователя нет транзакций, предлагает добавить первые расходы и создать первую цель накоплений.	Start tracking your first expenses.

10. Заключение

Третий блок заданий значительно усилил Kopilkin. Теперь в проекте есть более понятная микросервисная архитектура, persistent service-owned data, Kafka-based event publishing, Redis caching и rate limiting, API Gateway с Nginx load balancing, отдельный recommender microservice и полный обновлённый набор из семи C4-диаграмм.